UT2 · Un pedido se sirve entero o se rechaza
Una función con arrays y callbacks anónimos

Desarrollo Web en Entorno Servidor · 2.º DAW · Curso 2026/2027

Departamento de Informática · IES Ramón Arcas Meca · Lorca

1. Función que debes programar

Implementa una única función que decida qué pedidos puede servir un almacén y devuelva el stock restante. Resuélvela con array_map, array_reduce y callbacks anónimos, sin bucles ni recursión.

```
function procesarPedidos(array $stock, array $pedidos): array
```

1.1 Reglas del problema

1. Procesa los pedidos en el orden recibido. Cada línea indica el código de un producto y la cantidad solicitada: [producto, unidades]. Un producto puede aparecer en varias líneas del mismo pedido.

2. Acepta un pedido solo si puedes servir todas sus líneas con el stock disponible en ese momento. Descuenta sus unidades únicamente si lo aceptas.

3. Si no puedes servirlo entero, recházalo sin consumir stock. Los pedidos posteriores siguen teniendo acceso a esas unidades.

4. Un producto que no figure en stock tiene disponibilidad 0. Conserva las claves originales de stock y no añadas productos desconocidos.

5. Devuelve aceptados y rechazados como listas de identificadores, en su orden de llegada, y stock con las existencias finales. No modifiques las entradas.

Los identificadores son enteros distintos; los productos, cadenas no numéricas; las unidades, enteros positivos. Cada pedido tiene al menos una línea y el stock contiene enteros no negativos. No se pide validar estos tipos. Si pedidos es [], las listas estarán vacías y el stock se conservará.

2. Pasos para construir la función

Realiza las dos fases siguientes dentro de procesarPedidos. Los ejemplos muestran los datos que debes obtener; escribe tú los callbacks que los producen.

2.1 Obtén la demanda total de cada pedido

1. Usa array_map para transformar cada pedido: conserva su id y sustituye lineas por un array demanda, indexado por producto.

2. Dentro de esa transformación, usa array_reduce con acumulador inicial [] para sumar las cantidades de las líneas que correspondan al mismo producto.

Por ejemplo, este pedido:

```
[
    'id' => 102,
    'lineas' => [['A', 2], ['B', 1], ['A', 2]]
]
```

Debe convertirse en:

```
[
    'id' => 102,
    'demanda' => ['A' => 4, 'B' => 1]
]
```

El resultado de esta fase será un array de pedidos con id y demanda, en el mismo orden. En esta fase todavía no se descuenta stock.

2.2 Decide los pedidos y actualiza el stock

1. Recorre las demandas con array_reduce. Su acumulador guardará las listas de aceptados y rechazados y el stock disponible tras cada decisión.

Para el stock del ejemplo, el acumulador empieza así:

```
[
    'aceptados' => [],
    'rechazados' => [],
    'stock' => ['A' => 5, 'B' => 3, 'C' => 0]
]
```

2. Antes de descontar, comprueba que alcanza el stock actual para todos los productos del pedido. Puedes usar array_all; un producto desconocido tiene disponibilidad 0.

3. Si puedes servirlo entero, calcula el nuevo stock con array_map, conserva sus claves y añade el id a aceptados. Si no puedes, añade el id a rechazados y conserva el stock.

4. Devuelve el acumulador actualizado desde el callback para que el siguiente pedido reciba las listas y las existencias que deja el anterior.

Después de aceptar el pedido 101, cuya demanda es A: 2 y B: 1, el acumulador debe ser:

```
[
    'aceptados' => [101],
    'rechazados' => [],
    'stock' => ['A' => 3, 'B' => 2, 'C' => 0]
]
```

El pedido 102 se compara con ese stock: necesita 4 unidades de A y solo quedan 3. Se añade 102 a rechazados y el stock sigue siendo A: 3, B: 2, C: 0. El acumulador final será el retorno de procesarPedidos.

3. Ejemplo completo de entrada y salida

```
$stock = ['A'=>5, 'B'=>3, 'C'=>0];
$pedidos = [
    ['id'=>101, 'lineas'=>[['A',2], ['B',1]]],
    ['id'=>102, 'lineas'=>[['A',2], ['B',1], ['A',2]]],
    ['id'=>103, 'lineas'=>[['A',1], ['B',2]]],
    ['id'=>104, 'lineas'=>[['C',1]]],
    ['id'=>105, 'lineas'=>[['A',2]]],
    ['id'=>106, 'lineas'=>[['D',1]]],
];
```

La llamada procesarPedidos($stock, $pedidos) debe devolver:

```
[
    'aceptados' => [101, 103, 105],
    'rechazados' => [102, 104, 106],
    'stock' => ['A'=>0, 'B'=>0, 'C'=>0],
]
```